Parallel and Distributed Programming

Course Structure & Content

Theoretical Foundations

- Introductory concepts: architectures, concurrency, parallelism

- Steps in developing parallel programs
- Performance evaluation & parallel programming models
- Shared memory vs. distributed memory paradigms
- Design patterns for parallel and distributed systems

Practical Implementation

- Java threads (low-level API) & C++ (>=C++11) threads
- High-level APIs for multithreading
- Futures (Java &C++)
- CUDA (C++)
- MPI – Message Passing Interface (C/C++ examples)

Evaluation

- (35%) NL-> Laboratory work
- (5%) NS-> In-class work
- (10%) NM-> Test MPI
- (15%) NP -> Practical exam (week 14)
- (35%) NE -> Theoretical Examen

Final grade= [NL*35+NS*5+ NE*35+NM*10+NP*15]/100

- iff NE>4.5 && NL>4.5

References

- Ian Foster. Designing and Building Parallel Programs, Addison-Wesley 1995.
- Berna L. Massingill, Timothy G. Mattson, and Beverly A. Sanders,Addison A Pattern Language for Parallel Programming. Wesley Software Patterns Series, 2004.
- Michael McCool, Arch Robinson, James Reinders, Structured Parallel Programming: Patterns for Efficient Computation," Morgan Kaufmann,, 2012 .
- D. Culler, J. Pal Singh, A. Gupta. Parallel Computer Architecture: A Hardware/Software Approach. Morgan Kaufmann. 1998.
- Grama, A. Gupta, G. Karypis, V. Kumar. Introduction to Parallel Computing, Addison Wesley, 2003.
- D. Grigoras. Calculul Paralel. De la sisteme la programarea aplicatiilor. Computer Libris Agora, 2000.
- V. Niculescu. Calcul Paralel. Proiectare si dezvoltare formala a programelor paralele. Presa Univ. Clujana, 2006.
- B. Wilkinson, M. Allen, Parallel Programming Techniques and Applications Using Networked Workstations and Parallel Computers, Prentice Hall, 2002
- A. Williams. C++ Concurrency in Action PRACTICAL MULTITHREADING. Manning Publisher.2012.
- Tutoriale Java: http://docs.oracle.com/javase/tutorial/essential/concurrency/further.html
- C++ http://en.cppreference.com/w/
- OpenMP: http://openmp.org/
- MPI: http://www.mpi-forum.org/

What is Parallel Processing?

A **parallel computer** is a system that uses multiple processing elements simultaneously in a cooperative manner to solve a computational problem.

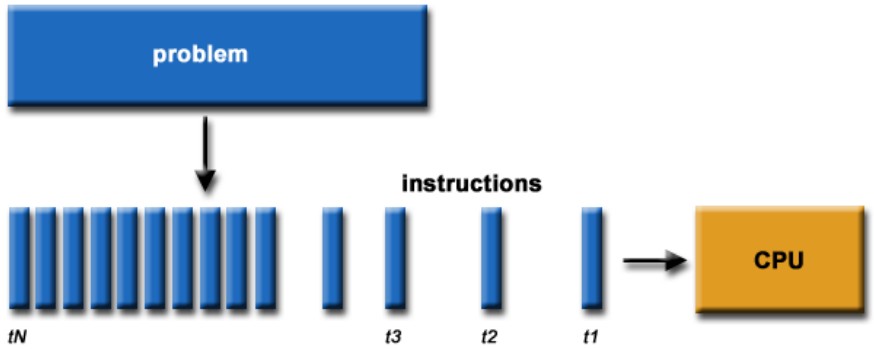
Parallel processing includes techniques and technologies making parallel computing possible
- hardware, networks, operating systems, libraries, compilers, and algorithms.

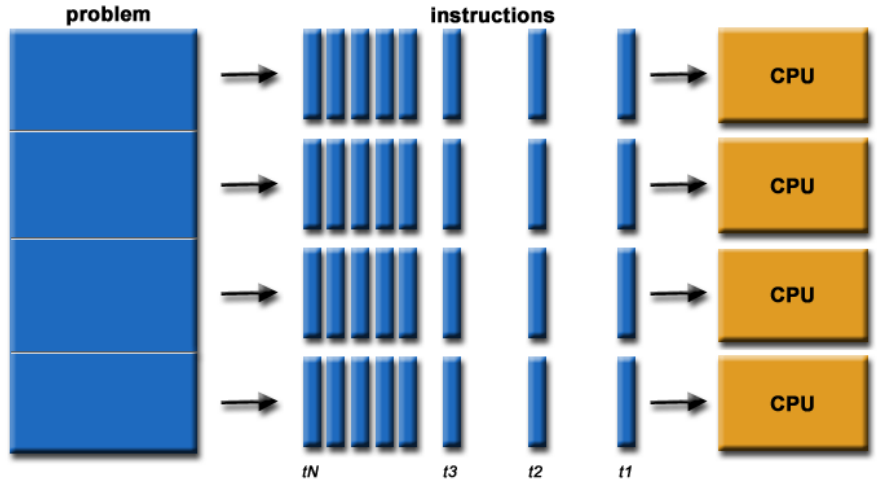
PERFORMANCE

"Parallelism is very much about performance!"

Serial vs. Paralel computation

(images from **Introduction to Parallel Computing** *Blaise Barney*)



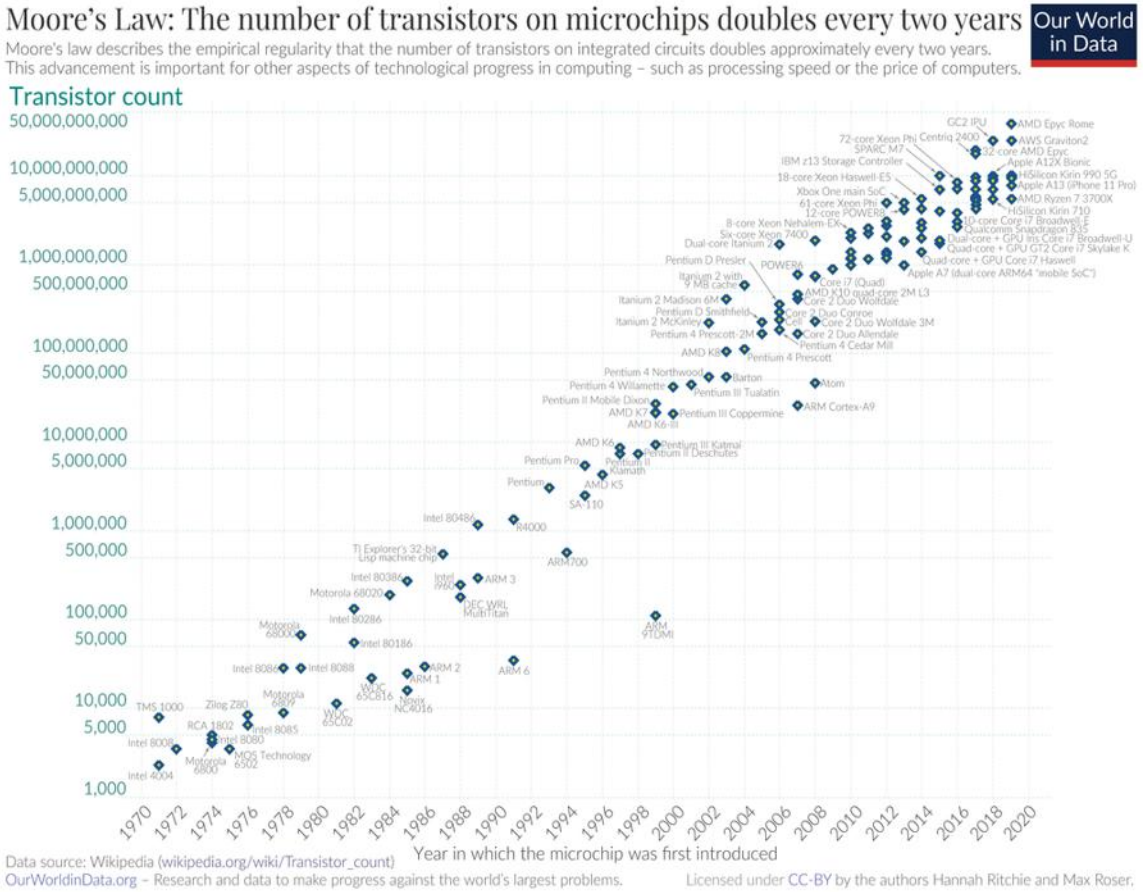


“It would appear that we have reached the limits
of what it is possible to achieve with computer technology,
although one should be careful with such statements,
as they tend to sound pretty silly in 5 years. “

(John von Neumann, 1949)

Limits of Serial Programming & Moore's Law

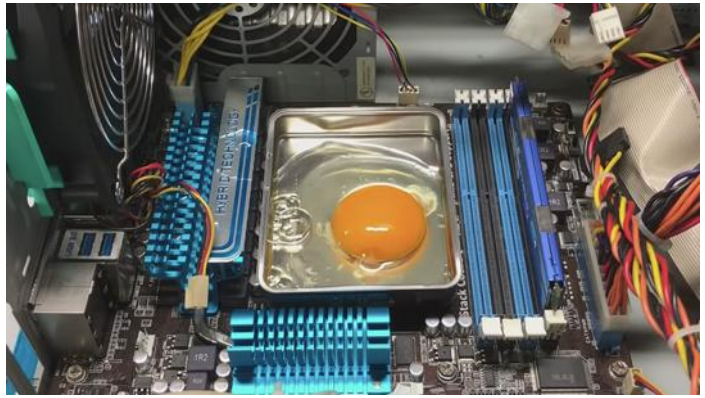
- **Transmission speed:** Speed of light (30 cm/ns), copper wire limit (9 cm/ns).
- **Miniaturization limit:** Number of transistors per chip.
- **Moore's Law:** Transistor count on integrated circuits doubles approximately every two years.
- **Economic limitations:** High R&D and manufacturing cost



Historical Shift: Clock Frequency vs. Multi-Core

- Traditional performance growth relied on increasing CPU clock frequency ("Riding Moore's law").
- **Major bottleneck:** Extreme chip heating! Higher clock frequency implies much higher power consumption.

  "Frying an egg on a computer."
- **Modern Solution:** Adding multiple cores instead of raising clock frequency to achieve desired performance without thermal overload.



Levels of Parallelism

1. Job-Level

Between independent jobs or distinct phases of job execution.

2. Program-Level

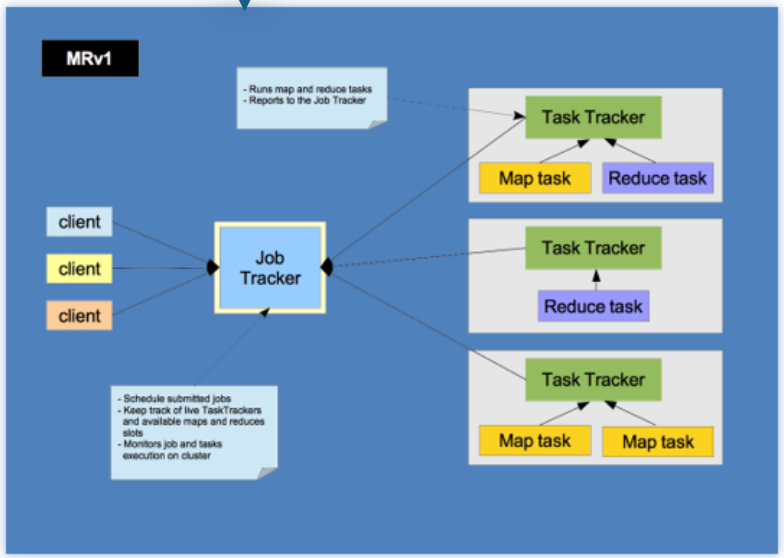
Between program parts, threads, or concurrent loop iterations.

3. Instruction-Level (ILP)

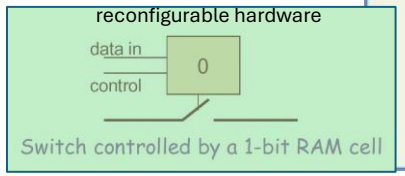
Overlapping execution of multiple instructions via compiler and processor.

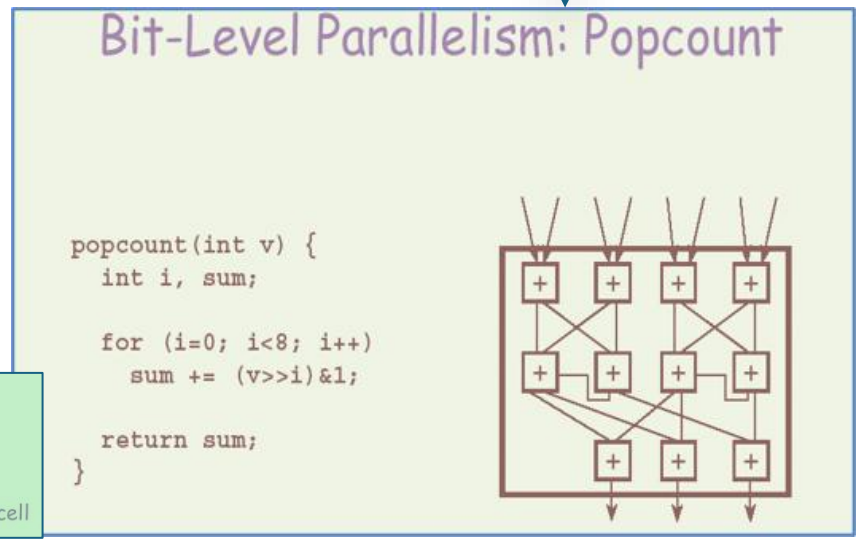
4. Bit & Arithmetic

Vector operations and arithmetic logic unit internal circuits.



ILP allows the compiler and the processor to overlap the execution of multiple instructions or even to change the order in which instructions are executed.





Processor Performance Metrics

- **IPC:** Number of instruction issues per machine cycle.
- **Instructions per second:** IPC × clock rate (cycles per second in Hertz).
- **Floating-point operations per second (FLOPS):** Core throughput metric.
- **System dependency:** Performance heavily relies on software and overall machine configuration (e.g. memory hierarchy, etc).

Improvement Methods

- Multiple execution units
- Pipelined instructions
- Multi-core architectures

Parallelism vs. Concurrency

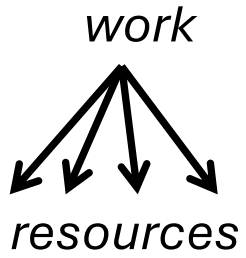
Concurrency

Managing multiple tasks making progress over overlapping time periods. Deals with structure, scheduling, and efficient access to shared resources.

Parallelism

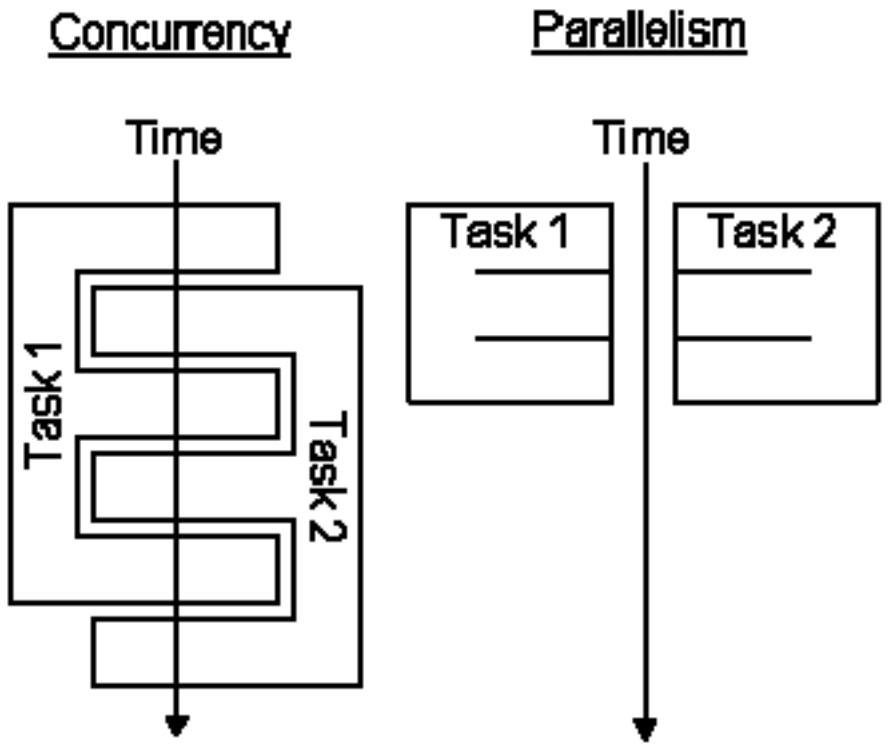
Executing multiple tasks literally at the same exact time. Requires multiple hardware computing resources.





Parallelism = Concurrency + Hardware Support

Simplistic approach - (*debatable)*



http://www.java-programming.info/tutorial/pdf/java/11-Java-Multithreaded-Programming.pdf

Why Use Parallel Programming?

Primary Motivations

- Faster computation (reduced response time)
- Solving larger computational problems in reasonable time

Secondary Motivations

- Effective utilization of compute resources
- Cost reduction
- Overcoming serial memory constraints

Grand Challenge Problems & Big Data

Grand Challenge Problems

Fundamental simulations in climate modeling, genomics, protein folding, astrophysics, and computational fluid dynamics requiring PetaFLOPS.

(en.wikipedia.org/wiki/Grand_Challenge)

New Data-Intensive Apps

Web search, data mining, sensor streams, and telescope sky scanning generating massive throughput demands.

Distributed Compute Projects

- **SETI@home:** Over 330,000 computers (~528 TeraFLOPS)
- **Folding@home:** Over 340,000 computers (~4.2 PetaFLOPS)

Grand Challenge Problems

- The solution or simulation of fundamental problems in science and engineering, with a strong scientific and economic impact, known as Grand Challenge Problems (GCPs), have been the driving force for Parallel Computing.
- Typically, GCPs simulate phenomena that cannot be measured by experimentation:

✓I Global Climate modeling

✓I Biology: genomics; protein folding, drug design

✓I Astrophysical modeling

✓I Computational Chemistry

✓I Earthquake and structural modeling

✓I Computational fluid dynamics (airplane design)

✓I Crash simulation

✓I Financial and economic modeling

New Data-Intensive Applications

- Currently, large volumes of data data are produced, and their processing and analysis also require high performance computing.
- Some examples:

✓ Data mining
✓ Web search
✓ networked video
✓ Video games and virtual reality
✓ Computer aided medical diagnosis

- Similarly, data is collected and stored at enormous speeds(GByte/hour).
- Some examples:

✓ sensor data streams
✓ telescope scanning the skies
✓ micro-arrays generating gene expression data

Parallel vs. Distributed Systems

Parallel Systems

- Tightly coupled shared memory (UMA, NUMA)
- Fast communication networks
- Unique global control
- Primary focus: Maximum performance

Distributed Systems

- Distributed memory with message passing
- Loosely coupled networks (Ethernet, etc.)
- No global clock (asynchronous)
- Primary focus: Scalability, reliability, resource sharing

- PARALLEL COMPUTERS **cooperate to solve difficult and complex problem**
- DISTRIBUTED COMPUTERS **individual goals and private activities**

Generaly ...

Parallel applications

Intent: Solve problems faster and more efficient

Distributed applications

Intent: solve specific problems:

- **COMMUNICATION SERVICES**
  ROUTING
  BROADCASTING

- **MAINTENANCE OF CONTROL STUCTURE**
  TOPOLOGY UPDATE
  LEADER ELECTION

- **RESOURCE CONTROL ACTIVITIES**
  LOAD BALANCING
  MANAGING GLOBAL DIRECTORIES

Distributed Applications

- Applications that are inherently distributed
  - – Distributed databases, airline ticket reservations, etc.; banking systems
- Sharing Information among users
- Resource sharing
- Reliability
- Scalability
  - – Loosely coupled connection; hot plugging
- Flexibility
  - - System reconfiguration to meet requirements

Performance& Scalability of distributed systems

- Unlike parallel systems, distributed systems involve:
  - Slower data transfer environment (a slower network)
  - Heterogeneity
- Solutions:
  - Batch processing of messages
    - Avoids involving the operating system for each message transfer.
  - Data caching
    - Avoids repeatedly transferring the same data.
  - Avoidance of centralized entities and algorithms
    - Avoids network saturation.
  - Performing “post” operations on the client side
    - Avoids heavy traffic between clients and servers.

Security

- Unlike parallel systems, in distributed systems:
  - There is no single central point of control

- Challenges & Solutions
  - Messages: stolen, modified, copied, etc.

    → *Solution:* Use of Cryptography
  - Failures**:** Node or link breakdowns
    - → *Solution:* Fault Tolerance solutions

Parallel vs. Distributed Systems

(from M. FUKUDA CSS434 System Models)

	Parallel Systems	**Distributed Systems**
Memory	**Tightly coupled shared** memory UMA, NUMA	**Distributed** memory Message passing, RPC, and/or used of distributed shared memory
Control	**Global clock** control SIMD, MIMD	**No global clock** control Synchronization algorithms needed
Processor interconnection	Order of **Tbps or more** Bus, mesh, tree, mesh of tree, and hypercube (-related) network	Order of **Gbps** Ethernet(bus), token ring and SCI (ring), myrinet(switching network)
Main focus	**Performance** Ex. - Scientific computing	Performance(**cost and scalability**) **Reliability/availability** **Information/resource sharing**